NUS HACKERS PROBLEM STATEMENT

AUTONOMOUS SG 2026 | ACACIA HACKATHON SOCIETY X NUS PRODUCT CLUB | UPDATED 31 OCT 2026

(Official) Problem Statement

We interact with countless things in our lives. Don't you ever hope things are less expensive, look better, and overall just work better? Why not reverse engineer something cool, useful and open source for it. Pick an opaque app, service, device, or system. Figure out how it works, then use what you uncover to build something even better.

Context

many things are $$$$$$$$$ but tokens r cheap!
why not reverse engineer something cool useful and open source

criteria:

how expensive old thing is now
how many people use expensive old thing
how cool ur new thing is
how u make expensive thing cheap
how open source is ur license even bro?

What to submit

- A public repository with an explicit OSI-approved open-source licence (e.g. MIT, Apache-2.0, GPL-3.0) in a LICENSE file.
- A live demo of the working build, under 3 minutes.
- A short README: the target, what you uncovered, the before/after cost with sources, and how to run it.

Rules

- **Stay legal.** Only reverse engineer things you own or can lawfully access. Do not break DRM or copy protection, bypass paywalls or authentication, or access accounts, servers or data that are not yours.

- **Clean-room rebuild.** Do not copy the original's proprietary code, assets, firmware or trademarks into your repo. Observing behaviour, public APIs, traffic from your own sessions and public documentation is fine.
- **No personal data.** Do not collect or publish other people's data. Remove your own tokens and keys before pushing.
- **Built today.** Write the new build during the sprint (10:15 AM to 4:15 PM SGT). Judges check commit history. Open-source libraries are fine; a pre-built project is not.
- **Source your numbers.** Prices and user counts need a link or screenshot. Unsourced claims score as weak.
- **Compare like for like.** Per user per month or per device, including running costs such as API tokens and hosting.
- **Licence check.** Your licence must be compatible with every library you include.
- Unsure whether a target is allowed? Ask an organiser before starting.

Judging Rubric

NUS Hackers

Five criteria. Score each from 1 to 5; apply the weights below. Total weight: 100%.

Cost of the status quo 20%	Reach of the old thing 15%	Coolness of the new build 25%
Cost reduction 25%	Open source licensing 15%	

Cost of the status quo

20% | 1-5

"how expensive old thing is now"

How expensive is the existing thing right now?

Name the app, service, device or system and what it costs people today in money, time or lock-in. Judges want a specific, priced target backed by evidence, not a general complaint that things cost too much.

STRONG
A named product with a credible, sourced price or cost of ownership.

WEAK
A vague "things are expensive" claim with no specific target or figure.

Reach of the old thing

15% | 1-5

"how many people use expensive old thing"

How many people actually use the expensive old thing?

Show how widely the target is used: user numbers, downloads, market share or another credible signal. Reverse engineering something millions rely on beats a tool almost nobody uses.

STRONG
Credible usage figures or clear evidence of a large user base.

WEAK
Assumed popularity with no numbers or source behind it.

Coolness of the new build

25% | 1-5

"how cool ur new thing is"

How cool is the new thing you built?

Show a working build that recreates or improves on the original, demoed live. Judges reward technical ambition, polish and the factor that makes people say they can't believe it works.

STRONG
A working demo that does something genuinely impressive or better than the original.

WEAK
Slides, mock-ups or a pitch with no working build behind it.

Judging Rubric

NUS Hackers | Criteria and scoring guidance

Cost reduction

25% | 1-5

"how u make expensive thing cheap"

How much cheaper did you make it?

Quantify the before and after. Compare your version's cost with the original's price, subscription or hardware cost, and show the saving per user. Free or near-free scores highest.

STRONG

A clear before/after number, ideally free or near-free for the user.

WEAK

Claimed savings with no numbers or comparison behind them.

Open source licensing

15% | 1-5

"how open source is ur license even bro?"

How open is the license, really?

Point to a public repository with an explicit open-source license. The code should be public and reusable by anyone, with the license clearly stated in the repo.

STRONG

A public repo with an explicit open-source license and reusable code.

WEAK

No license, unclear terms, or code that is not public at all.

SCORING AND JUDGES' NOTES

Scale: 1 = weak; 2 = limited; 3 = credible; 4 = strong; 5 = exceptional evidence.

Weighted total / 100 = sum of (criterion score / 5 x criterion weight). Example: coolness of the new build scored 4/5 earns 20 of its 25 points.

Score criteria separately. Ask every team: "What did you uncover, and how do you know it works?"
Reward working builds and real numbers, not presentation polish.